

1 Problem Set

Problem 2-35: Verify Equation (2.67).

$$H^{-1} = \begin{bmatrix} R^T & -R^T d \\ 0 & 1 \end{bmatrix}$$

If $H = \begin{bmatrix} R & d \\ 0 & 1 \end{bmatrix}$, we can verify its inverse Equation (2.67) by proving the dot product of the equations equals an identity matrix.

$$\begin{aligned} H \cdot H^{-1} &= \begin{bmatrix} R & d \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R^T & -R^T d \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} RR^T & -RR^T d + d \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} I & 0 \\ 0 & 1 \end{bmatrix} \\ &= I \end{aligned}$$

Problem 2-36: Compute the homogenous transformation representing a translation of 3 units along the x-axis followed by a rotation of $\frac{\pi}{2}$ about the current z-axis followed by a translation of 1 unit along the fixed y-axis. Sketch the frame. What are the coordinates of the origin o_1 with respect to the original frame in each case?

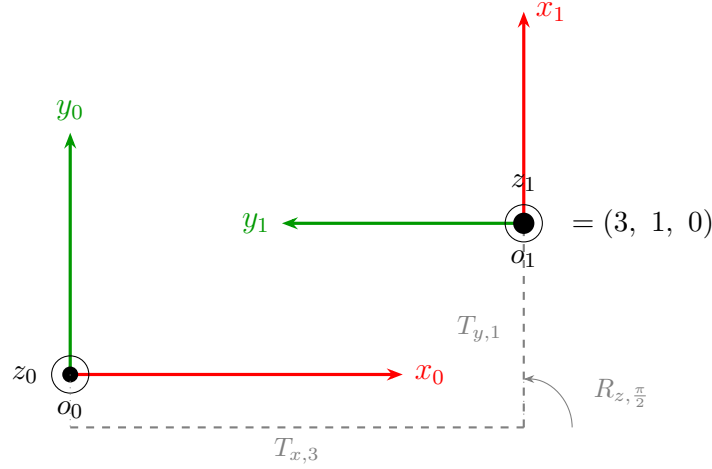
While taking account of the order of matrix multiplication, current frames are post-multiplied and fixed frames are pre-multiplied. With that, we have:

$$H = T_{y,1} \cdot T_{x,3} \cdot R_{z,\frac{\pi}{2}}$$

The individual matrices are:

$$T_{x,3} = \begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_{z,\frac{\pi}{2}} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T_{y,1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Sketching it out, we have:



Computing $T_{x,3} \cdot R_{z, \frac{\pi}{2}}$:

$$T_{x,3} \cdot R_{z, \frac{\pi}{2}} = \begin{bmatrix} 0 & -1 & 0 & 3 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Then pre-multiplying by $T_{y,1}$:

$$H = \begin{bmatrix} 0 & -1 & 0 & 3 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The origin o_1 expressed in the base frame is the translation vector:

$$o_1 = \begin{bmatrix} 3 \\ 1 \\ 0 \end{bmatrix}$$

Problem 2-37: Consider the diagram of Figure 2.13. Find the homogenous transformations H_1^0 H_2^0 H_2^1 representing the transformations among the three frames shown. Show that $H_2^0 = H_1^0 H_2^1$.

Frame 1 is 1 unit above Frame 0 along z_0 , with axes: $x_1 = -z_0$, $y_1 = y_0$, $z_1 = x_0$.

Frame 2 is 1 unit along y_0 from the origin, with axes: $x_2 = -y_0$, $y_2 = z_0$, $z_2 = -x_0$.

$$H_1^0 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad H_2^0 = \begin{bmatrix} 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

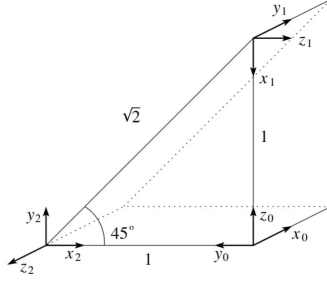


Figure 2.13: Diagram for Problem 2-37.

Inverse of H_1^0 :

Using $H^{-1} = \begin{bmatrix} R^T & -R^T d \\ 0 & 1 \end{bmatrix}$ with $d = [0, 0, 1]^T$: $R^T d = [-1, 0, 0]^T$, so $-R^T d = [1, 0, 0]^T$.

$$(H_1^0)^{-1} = \begin{bmatrix} 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformation H_2^1 :

$$\begin{aligned} H_2^1 &= (H_1^0)^{-1} \cdot H_2^0 \\ &= \begin{bmatrix} 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & -1 & 0 & 1 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Verification that $H_2^0 = H_1^0 \cdot H_2^1$:

$$H_1^0 \cdot H_2^1 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 & 1 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = H_2^0 \quad \checkmark$$

Problem 2-38: Consider the diagram of Figure 2.14. A robot is set up 1 meter from a table. The table top is 1 meter high and 1 meter square. A frame $o_1x_1y_1z_1$ is fixed to the edge of the table as shown. A cube measuring 20 cm on a side is placed in the center of the table with frame $o_2x_2y_2z_2$ established at the center of

the cube as shown. A camera is situated directly above the center of the block 2 meters above the table top with frame $o_3x_3y_3z_3$ attached as shown. Find the homogenous transformation relating the frame $o_2x_2y_2z_2$ to the camera frame $o_3x_3y_3z_3$.

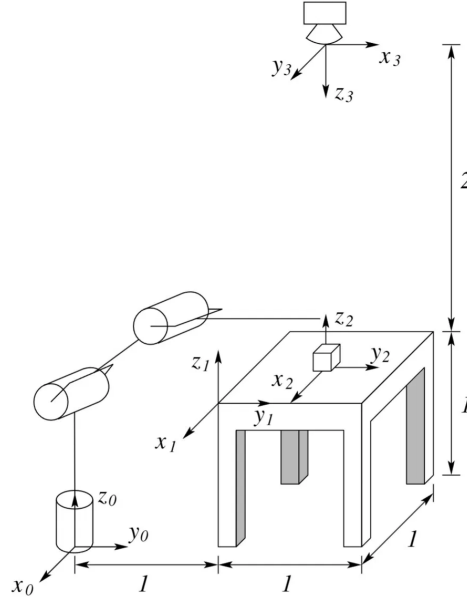


Figure 2.14: Diagram for Problem 2-38.

From Figure 2.14, we identify the following frames relative to Frame 0 (robot base). The robot is 1m from the table edge, the table is 1m high and 1m square, the cube is 20cm on a side (so its center is 0.1m above the table), and the camera is 2m above the table top.

- Frame 1: table edge, translated 1m along y_0 and 1m up along z_0 , same orientation as Frame 0
- Frame 2: cube center, offset -0.5m along x_0 , 1.5m along y_0 , and 1.1m along z_0 (1m table + 0.1m half-cube height), same orientation as Frame 0
- Frame 3: camera, directly above cube center at 3m height, different orientation to Frame 0

Homogeneous transformations relative to Frame 0:

$$H_1^0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad H_2^0 = \begin{bmatrix} 1 & 0 & 0 & -0.5 \\ 0 & 1 & 0 & 1.5 \\ 0 & 0 & 1 & 1.1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad H_3^0 = \begin{bmatrix} 0 & 1 & 0 & -0.5 \\ 1 & 0 & 0 & 1.5 \\ 0 & 0 & -1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Finding $H_2^3 = (H_3^0)^{-1} \cdot H_2^0$:

The inverse of Frame 3 simplifies to:

$$(H_3^0)^{-1} = \begin{bmatrix} 0 & 1 & 0 & -1.5 \\ 1 & 0 & 0 & 0.5 \\ 0 & 0 & -1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$H_2^3 = (H_3^0)^{-1} \cdot H_2^0 = \begin{bmatrix} 0 & 1 & 0 & -1.5 \\ 1 & 0 & 0 & 0.5 \\ 0 & 0 & -1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -0.5 \\ 0 & 1 & 0 & 1.5 \\ 0 & 0 & 1 & 1.1 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 1.9 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The translation vector $(0, 0, 1.9)^T$ confirms that the cube center lies directly below the camera at a distance of $2 - 0.1 = 1.9$ meters, which is consistent with the geometry in Figure 2.14.

Problem 2-42: In general, multiplication of homogenous transformation matrices is not commutative. Consider the matrix product

$$H = R_{x,\alpha} \cdot T_{x,b} \cdot T_{z,d} \cdot R_{z,\theta}$$

Determine which pairs of the four matrices on the right-hand side commute. Explain why these pairs commute. Find all permutations of these four matrices that yield the same homogenous transformation matrix, H .

The four matrices are:

$$R_{x,\alpha} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha & 0 \\ 0 & s_\alpha & c_\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_{x,b} = \begin{bmatrix} 1 & 0 & 0 & b \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{z,d} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_{z,\theta} = \begin{bmatrix} c_\theta & -s_\theta & 0 & 0 \\ s_\theta & c_\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Commuting Pairs:

- $R_{x,\alpha}$ and $T_{x,b}$: a rotation about the x -axis and a translation along the x -axis act independently, since both operate purely along the x -axis.
- $T_{x,b}$ and $T_{z,d}$: pure translations always commute regardless of direction, as they only affect the translation vector and do not interact.
- $T_{z,d}$ and $R_{z,\theta}$: a translation along the z -axis and a rotation about the z -axis act independently, since the z -component is unaffected by rotation about z .

- All other pairs **do not commute**: specifically, $R_{x,\alpha}$ does not commute with $T_{z,d}$ or $R_{z,\theta}$, and $T_{x,b}$ does not commute with $R_{z,\theta}$.

Valid Permutations:

The three commuting pairs define a chain:

$$R_{x,\alpha} \leftrightarrow T_{x,b} \leftrightarrow T_{z,d} \leftrightarrow R_{z,\theta}$$

where each $\leftrightarrow$ denotes a pair that may be swapped freely. However, these swaps are *not* independent: swapping $T_{x,b}$ and $T_{z,d}$ (the middle pair) separates $T_{z,d}$ from $R_{z,\theta}$, making that swap unavailable in the resulting ordering. Exhausting all valid adjacent swaps yields exactly five permutations:

1. $R_{x,\alpha} \cdot T_{x,b} \cdot T_{z,d} \cdot R_{z,\theta}$ (original)
2. $T_{x,b} \cdot R_{x,\alpha} \cdot T_{z,d} \cdot R_{z,\theta}$ (swap $R_{x,\alpha} \leftrightarrow T_{x,b}$)
3. $R_{x,\alpha} \cdot T_{x,b} \cdot R_{z,\theta} \cdot T_{z,d}$ (swap $T_{z,d} \leftrightarrow R_{z,\theta}$)
4. $T_{x,b} \cdot R_{x,\alpha} \cdot R_{z,\theta} \cdot T_{z,d}$ (swap both endpoints)
5. $R_{x,\alpha} \cdot T_{z,d} \cdot T_{x,b} \cdot R_{z,\theta}$ (swap $T_{x,b} \leftrightarrow T_{z,d}$)

No further reorderings are possible: once $T_{x,b}$ and $T_{z,d}$ are swapped (permutation 5), $T_{z,d}$ is no longer adjacent to $R_{z,\theta}$ and cannot be swapped with it; $T_{x,b}$ is no longer adjacent to $R_{x,\alpha}$ and cannot be swapped with it; and $R_{x,\alpha}$ is now adjacent to $T_{z,d}$, with which it does *not* commute. All other arrangements therefore produce a different H .

2 ROS 2 Assignment

2.1 Overview

This package implements a ROS 2 node that subscribes to a `/quaternion` topic carrying a `Float32MultiArray` message with four values $[w, x, y, z]$ and converts the quaternion to ZYX Euler angles (roll ϕ , pitch θ , yaw ψ). The conversion uses the standard formulas from the Wikipedia “Conversion between quaternions and Euler angles” article, with a gimbal-lock guard on the pitch term. The resulting angles are printed to the terminal in both radians and degrees each time a new quaternion is received.

2.2 Package Structure

```
quat_to_euler/  
├── quat_to_euler/  
│   ├── __init__.py  
│   └── quat_to_euler_node.py  
├── package.xml  
├── setup.py  
└── setup.cfg
```

2.3 Node Implementation

```
1 import rclpy  
2 from rclpy.node import Node  
3 from std_msgs.msg import Float32MultiArray  
4 import math  
5  
6 class QuatToEulerNode(Node):  
7     def __init__(self):  
8         super().__init__('quat_to_euler')  
9         self.subscription = self.create_subscription(  
10             Float32MultiArray,  
11             'quaternion',  
12             self.listener_callback,  
13             10)  
14         self.get_logger().info('Quaternion to Euler node started.')  
15         self.get_logger().info('Waiting for quaternion input [w, x, y, z]...')  
16  
17     def listener_callback(self, msg):  
18         if len(msg.data) != 4:  
19             self.get_logger().error(  
20                 f'Expected 4 values [w, x, y, z], got {len(msg.data)}')  
                )
```

```

21         return
22
23     w, x, y, z = msg.data
24
25     norm = math.sqrt(w**2 + x**2 + y**2 + z**2)
26     if abs(norm - 1.0) > 0.01:
27         self.get_logger().warn(
28             f'Quaternion norm is {norm:.4f}, expected 1.0.'
29             f'Results may be inaccurate.')
30
31     sinr_cosp = 2.0 * (w * x + y * z)
32     cosr_cosp = 1.0 - 2.0 * (x * x + y * y)
33     phi = math.atan2(sinr_cosp, cosr_cosp)
34
35     sinp = 2.0 * (w * y - z * x)
36     if abs(sinp) >= 1.0:
37         theta = math.copysign(math.pi / 2, sinp)
38     else:
39         theta = math.asin(sinp)
40
41     siny_cosp = 2.0 * (w * z + x * y)
42     cosy_cosp = 1.0 - 2.0 * (y * y + z * z)
43     psi = math.atan2(siny_cosp, cosy_cosp)
44
45     phi_deg = math.degrees(phi)
46     theta_deg = math.degrees(theta)
47     psi_deg = math.degrees(psi)
48
49     self.get_logger().info(
50         f'\nReceived quaternion: w={w}, x={x}, y={y}, z={z}\n'
51         f'Roll(phi)={phi:.4f} rad = {phi_deg:.2f} deg\n'
52         f'Pitch(theta)={theta:.4f} rad = {theta_deg:.2f} deg\n'
53         f'Yaw(psi)={psi:.4f} rad = {psi_deg:.2f} deg')
54
55 def main(args=None):
56     rclpy.init(args=args)
57     node = QuatToEulerNode()
58     rclpy.spin(node)
59     node.destroy_node()
60     rclpy.shutdown()
61
62 if __name__ == '__main__':
63     main()

```


2.4 Conversion Formulas

The quaternion (w, x, y, z) is converted to Euler angles (ϕ, θ, ψ) using:

$$\begin{aligned}\phi &= \arctan 2(2(wx + yz), 1 - 2(x^2 + y^2)) \\ \theta &= \arcsin(2(wy - zx)) \\ \psi &= \arctan 2(2(wz + xy), 1 - 2(y^2 + z^2))\end{aligned}$$

2.5 How to Run

```
ros2 run quat_to_euler quat_to_euler_node
```

Publish test quaternions from the terminal:

```
ros2 topic pub /quaternion std_msgs/msg/Float32MultiArray \
  "data: [1.0, 0.0, 0.0, 0.0]"
```

```
ros2 topic pub --once /quaternion std_msgs/msg/Float32MultiArray \
  "data: [0.7071, 0.0, 0.0, 0.7071]"
```

```
ros2 topic pub --once /quaternion std_msgs/msg/Float32MultiArray \
  "data: [0.7071, 0.7071, 0.0, 0.0]"
```

2.6 Results

```

tracy@motherboard:~/rbe500/ros2_ws$ ros2 run quat_to_euler quat_to_euler_node
[INFO] [1781452553.414183309] [quat_to_euler]: Quaternion to Euler node started.
[INFO] [1781452553.414496969] [quat_to_euler]: Waiting for quaternion input [w, x, y, z]...
[INFO] [1781452597.276699657] [quat_to_euler]:
Received quaternion: w=1.0, x=0.0, y=0.0, z=0.0
  Roll (phi)   = 0.0000 rad = 0.00 deg
  Pitch (theta) = 0.0000 rad = 0.00 deg
  Yaw (psi)    = 0.0000 rad = 0.00 deg
[INFO] [1781452605.801317670] [quat_to_euler]:
Received quaternion: w=0.707099974155426, x=0.0, y=0.0, z=0.707099974155426
  Roll (phi)   = 0.0000 rad = 0.00 deg
  Pitch (theta) = 0.0000 rad = 0.00 deg
  Yaw (psi)    = 1.5708 rad = 90.00 deg
[INFO] [1781452616.194995622] [quat_to_euler]:
Received quaternion: w=0.707099974155426, x=0.707099974155426, y=0.0, z=0.0
  Roll (phi)   = 1.5708 rad = 90.00 deg
  Pitch (theta) = 0.0000 rad = 0.00 deg
  Yaw (psi)    = 0.0000 rad = 0.00 deg

```

Figure 1: Terminal output showing Euler angle conversion